

DAG-Stream E-Com Analytics Engine

Index / Table of Contents

1. Aim and Objectives

2. Tools & Services Used

3. End-to-End Data Flow

4. Data Pipeline Architecture and Explanation

4.1 Data Ingestion (Raw Layer / Bronze)

4.2 Automated Pipeline Management (Cloud Composer)

4.3 Data Cleaning & Transformation (Silver Layer)

4.4 Analytics & Reporting (Gold Layer)

5. Pipeline Build & Execution Instructions

5.1 Create Cloud Storage Buckets & Folders

5.2 Set Up Pub/Sub & Storage Permissions

5.3 Provision Cloud Spanner Instance & Table

5.4 Run the Stream Publisher

5.5 Run Dataproc Serverless PySpark

5.6 Create Cloud Composer Environment & Deploy DAG

5.7 Create BigQuery External Connection & Dataset

5.8 Execute & Schedule Federated Analytics

5.9 Pipeline Data Verification

Aim and Objectives:

The main goal of this project is to build an automated, serverless data pipeline that processes high-volume e-commerce order streams into a medallion data lakehouse architecture on Google Cloud Platform (GCP).

Bronze Ingestion: Stream raw JSON order payloads into Cloud Storage via Google Cloud Pub/Sub.

Silver Transformation: Run an automated PySpark job on Dataproc Serverless to clean, flatten, deduplicate, and store analytical data in Parquet format while synchronizing transaction line items to Cloud Spanner.

Gold Analytics: Execute federated SQL queries in BigQuery via BigQuery External Connections to generate daily product sales aggregations without data movement overhead.

Orchestration: Automate and schedule the end-to-end workflow using Apache Airflow hosted on Cloud Composer.

Tools & Services Used

Google Cloud Pub/Sub: Ingests streaming raw e-commerce order payloads from event producers.

Google Cloud Storage (GCS): Provides scalable object storage for Bronze (raw JSON) and Silver (Parquet) layers.

Dataproc Serverless: Executes PySpark ETL jobs without requiring manually managed clusters.

Cloud Spanner: Serves as a globally distributed, relational database for real-time order line-item transactions.

BigQuery & BigQuery Omni/Connections: Runs federated queries directly over Cloud Spanner to populate Gold analytical datasets.

Cloud Composer (Apache Airflow): Manages task scheduling, dependencies, and execution workflow.

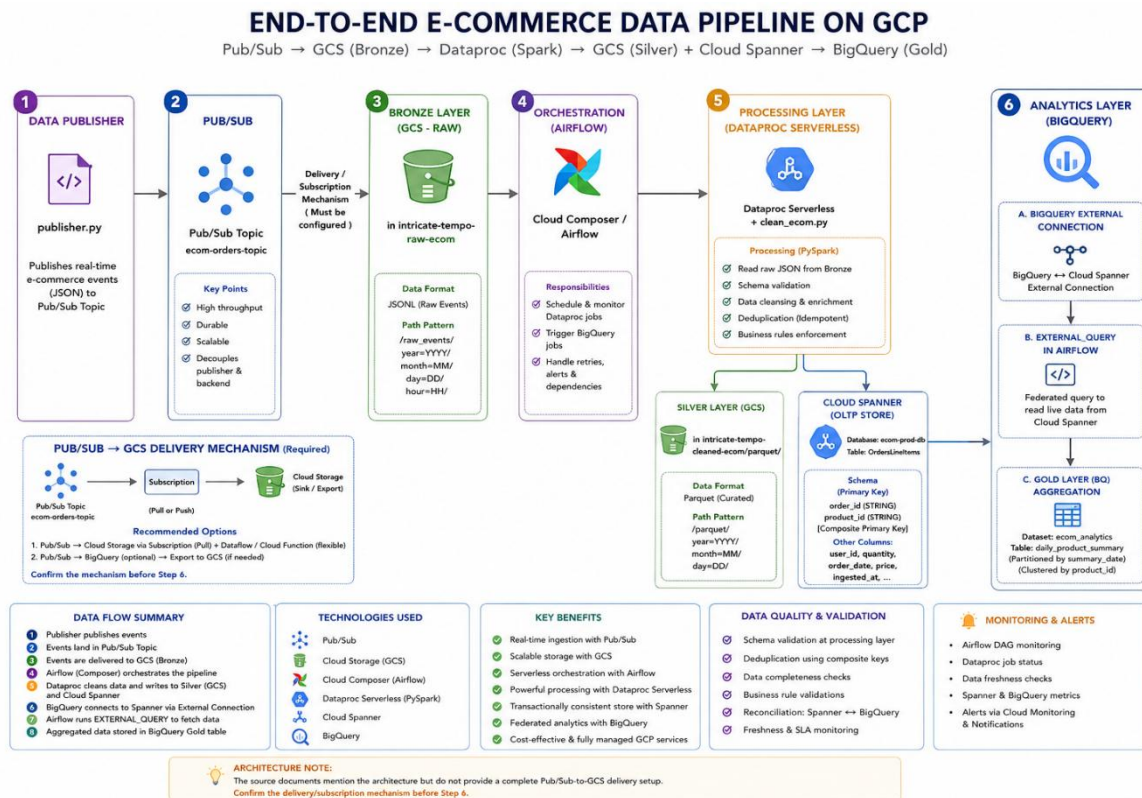
End-to-End Data Flow

Step 1 (Raw Payload Ingestion): An e-commerce event publisher generates raw order JSON payloads to Cloud Pub/Sub, which land directly into the Bronze storage bucket via a Cloud Storage Pub/Sub subscription.

Step 2 (Cleansing & Transformation): Airflow triggers a Dataproc Serverless PySpark job that cleans, flattens, and deduplicates the raw JSON records. It writes the processed Parquet files into the Silver storage bucket while simultaneously upserting transactional records into the Cloud Spanner database table.

Step 3 (Gold Layer Aggregation): Airflow executes a BigQuery task that queries the Spanner table directly using a federated SQL connection, aggregating daily sales metrics without data movement and materializing the results into the Gold analytics table.

Data Pipeline Architecture and Explanation:



1. Data Ingestion (Raw Layer / Bronze)

A Python script simulates live online shopping orders and sends them as messages to Google Cloud Pub/Sub.

Pub/Sub streams these raw JSON order files directly into a Cloud Storage bucket, which acts as the Bronze (raw) layer.

2. Automated Pipeline Management (Cloud Composer)

Cloud Composer (powered by Apache Airflow) manages and runs the entire data workflow automatically using two scheduled tasks.

3. Data Cleaning & Transformation (Silver Layer)

Task 1 (Dataproc Serverless): Airflow starts a PySpark job on Dataproc to read the raw JSON files. It removes duplicates, cleans missing values, and fixes data formats.

Saving Clean Data: The job splits the cleaned output into two places:

Saves optimized Parquet files into a Cloud Storage bucket (Silver layer) for long-term storage and analytical processing.

Updates live transaction records inside a Cloud Spanner database table (OrdersLineItems).

4. Analytics & Reporting (Gold Layer)

Task 2 (BigQuery Federated Query): Airflow triggers a BigQuery SQL query that connects directly to Cloud Spanner using a federated connection.

BigQuery reads the live data inside Cloud Spanner without copying or moving it around, calculates daily sales summaries, and saves the final insights directly into the Gold analytics table (daily_product_summary).

Pipeline Build & Execution Instructions:

Step 1: Create Cloud Storage Buckets & Folders

Set up Bronze and Silver storage layers.

In the GCP Console, navigate to **Cloud Storage > Buckets** and click + **Create**.

Bronze Bucket (Raw Data):

Name: pub-sub-dag-rawdata

Location type: Region → asia-south1 (Mumbai)

Leave other settings as default and click **Create**.

Silver Bucket (Cleaned Data & Scripts):

Name: pub-sub-cleandata-dag

Location type: Region → asia-south1 (Mumbai)

Click **Create**.

Inside pub-sub-clean-data, create two folders:

scripts/

parquet/

Step 2: Set Up Pub/Sub & Storage Permissions

Configure message broker and bucket-level IAM.

Go to **Pub/Sub > Topics** → Click + **Create Topic**.

Topic ID: ecom-raw-topic

Click **Create**.

Open ecom-raw-topic → Go to the **Subscriptions** tab → Click + **Create Subscription**.

Subscription ID: ecom-raw-gcs-sub

Delivery type: Select **Write to Cloud Storage**.

Cloud Storage bucket: Select pub-sub-rawdata.

Filename prefix: raw_order_

Output format: JSON.

Click **Create**.

Grant IAM Permissions:

Go to **Cloud Storage > Buckets > pub-sub-rawdata > Permissions** tab.

Click + **Grant access** (or Add Principals).

New principal: service-54903645206@gcp-sa-pubsub.iam.gserviceaccount.com

Assign two roles:

Storage Legacy Bucket Reader

Storage Object Creator

Click **Save**.

Step 3: Provision Cloud Spanner Instance & Table

Provision transactional OLTP database and schema.

Go to **Spanner > Instances** → Click + **Create Instance**.

Instance name / ID: ecom-spanner-instance

Configuration: Regional → asia-south1 (Mumbai)

Click **Create**.

Inside the instance, click + **Create Database**:

Database name: ecom-db

Dialect: Google Standard SQL

In the DDL definition window, execute: SQL Query provided in code documentation

Click **Create**.

Step 4: Run the Stream Publisher (publisher.py)

Generate mock streaming events into Pub/Sub.

Save the following code locally or in Cloud Shell as publisher.py: Take it from the code documentation

Run the script: `python3 publisher2.py`

Check **Cloud Storage** > **pub-sub-rawdata** to verify raw_order_... JSON files have landed.

Step 5: Run Dataproc Serverless PySpark (clean_ecom2.py)

Create clean_ecom.py containing the GCS Python SDK ingestion fix (bypassing Hadoop URI colon issues):

Take it from the code documentation

Upload the script to GCS:

```
gcloud storage cp clean_ecom2.py
```

```
gs://pub-sub-clean-data/scripts/clean_ecom2.py
```

Step 6: Create Cloud Composer Environment & Deploy DAG

Open [Cloud Composer](#) and click + **CREATE ENVIRONMENT** " → "Composer 2.

Name: ecom-composer-env

Region: asia-south1

Click **CREATE** (takes ~10-15 minutes to provision).

Once created, click on ecom-composer-env and open the **DAGs folder** link (which redirects to its underlying GCS bucket).

Create ecom_pipeline_dag.py locally with the code below and upload it to that **DAGs folder**:
Take ecom_pipeline_dag.py code from Code Documentation

Step 7: Create BigQuery External Connection & Dataset

Establish live connection from BigQuery to Spanner.

Create BigQuery Connection:

In BigQuery Studio, click + **Add Data > Connections to external data sources**.

Connection type: Cloud Spanner

Connection ID: spanner-ecom-conn

Location type: Region → asia-south1 (Mumbai)

Database name:

projects/snappy-mapper-498509-e0/instances/ecom-spanner-instance/databases/ecom-db

Click **Create connection**.

Create BigQuery Dataset:

In the Explorer pane, click the three dots (:) next to snappy-mapper-498509-e0

→ **Create dataset**.

Dataset ID: ecom_analytics

Location: Region → asia-south1 (Mumbai)

Click **Create dataset**.

Step 7: Execute & Schedule Federated Analytics

Materialize initial table and configure recurring run.

Create and Populate the Target Table (CTAS):

Run this in BigQuery Studio to create the table and insert the initial aggregated batch:

OPTIONAL:

```
CREATE TABLE IF NOT EXISTS `snappy-mapper-498509-e0.ecom_analytics2.daily_product_summary` AS SELECT
product_id,
SUM(quantity) AS total_items_sold, COUNT(DISTINCT order_id) AS total_orders, CURRENT_TIMESTAMP() AS aggregated_at
FROM EXTERNAL_QUERY(
"projects/snappy-mapper-498509-e0/locations/asia-south1/connections/spanner-ecom-conn",
"SELECT order_id, product_id, quantity FROM EcomOrders"
)
GROUP BY product_id;
```

Schedule the Incremental Job: Paste the append query in the editor:

```
INSERT INTO `snappy-mapper-498509-e0.ecom_analytics2.daily_product_summary` ( product_id,
total_items_sold, total_orders, aggregated_at
) SELECT
product_id,
SUM(quantity) AS total_items_sold, COUNT(DISTINCT order_id) AS total_orders, CURRENT_TIMESTAMP() AS aggregated_at
FROM EXTERNAL_QUERY(
"projects/snappy-mapper-498509-e0/locations/asia-south1/connections/spanner-ecom-conn",
"SELECT order_id, product_id, quantity FROM EcomOrders"
)
GROUP BY product_id;
```

Click **Schedule > Create new scheduled query**.

Display name: daily-spanner-sales-summary

Frequency: Daily

Location: asia-south1

Click **Save**.

Step 8: Pipeline Data Verification

Confirm data has loaded successfully across all layers.

Run this SQL query in BigQuery Studio to verify data population in the final analytics summary table:

Take the from the code documenation

